

Introduction

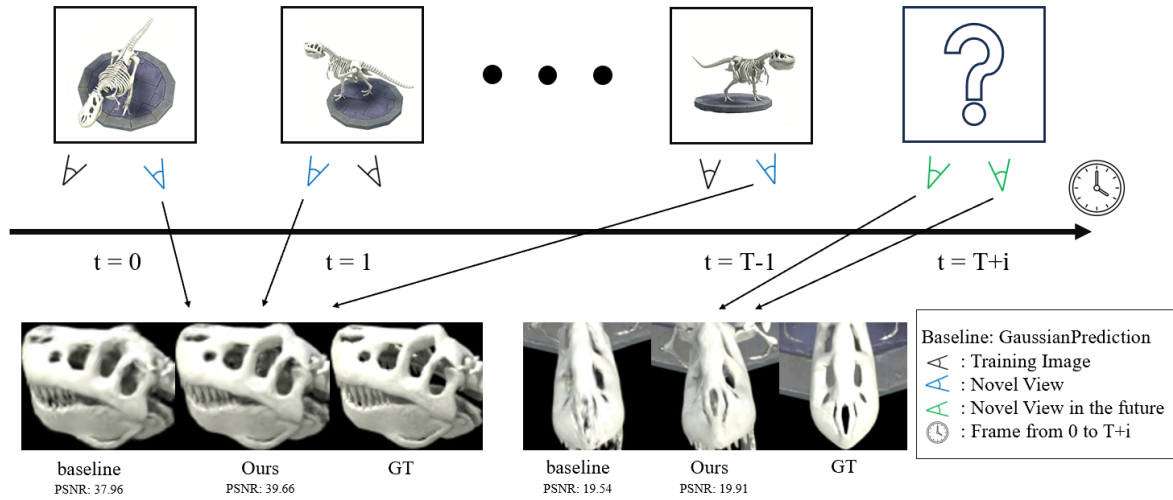


图1：本图解释了本文的基本任务以及与当前主流方法的结果对比。本文的2个基本任务是输入一组图片以及它们的时间（时间帧 $t \in [0, T - 1]$ ）和位姿（图中的黑色视点），输出在不同位姿（图中的蓝色视点）下渲染的新图片，以及在未来帧（时间帧 $t \in [T, T + i]$ ）的不同位姿（绿色视点）下的新图片。此外，我们还在下图中展示了与当前主流标杆方法GaussianPrediction在两个基本任务中结果对比，其中PSNR为峰值信噪比，越高代表渲染结果越接近GT。在渲染图中，我们可以发现，由于GaussianPrediction方法中的控制点可以自由移动且不受限制，导致其渲染结果在细节部分出现了伪影。而本文提出的OctGaussian方法以Octree作为场景划分的基本结构，不仅在新视图合成任务中，而且在未来帧的新视图合成任务中，其渲染结果的细节部分更加清晰。

近年来，自动驾驶、无人机等计算机视觉应用领域的快速发展，使得计算机对动态场景的构建能力和预测能力有了较高的要求，而且对于动态场景的重建和新视点的渲染往往需要非常苛刻要求。在当前的计算机视觉领域，在动态3D世界中依旧很难同时做到预测未来短时间内场景中各种物体的变化情况，以及从任意新的视点渲染未来的场景。

尽管如此，以往的大量工作仍然做出了许多努力。XXX在视角不改变的情况下，基于过去的观测来预测未来场景的变化。尽管其预测结果很好，但是在实际应用，视点往往不是固定的。在新视图合成领域，XXXX方法可以通过从不同的是视角下重建整个动态场景，然后再从任意选择的视角下渲染任意时刻的场景，但是其并不具备对未来的预测能力。XX方法可以从过去的一系列点云推断出未来点云的变化情况，这在需要路径规划的自动驾驶、无人机领域有着巨大的作用，然而它却无法渲染高质量的图像。

在3D高斯喷溅技术诞生后，其凭借快速的场景重建和实时级的渲染能力，迅速被拓展到计算机视觉的各个领域。对于动态场景重建和预测领域，GaussianPrediction提出使用控制点来重建动态场景，并使用图神经网络为未来场景进行预测，实现了高质量的新视图合成与未来场景预测，然而控制点在训练中可以自由漂移，容易出现场景重建和预测出现过拟合的情况。ODE-GS提出使用神经常微分方程来代替图神经网络对未来场景进行预测，实现了更加准确的控制点连续移动预测，然而该方法仍是基于控制点的空间划分方式，因此还是会出现场景覆盖不均匀的问题。

基于上述方法的各种限制，我们提出OctPrediction，一种基于3D高斯喷溅技术的动态场景重建与预测方法。给定不同时间下不同视点的观测图片，能够生成不同时间下不同视点的观测图片，甚至能够生成未来时间的不同视点的观测图片。如图1所示，我们需要输入一组图片以及它们的时间（时间帧 $t \in [0, T - 1]$ ）和位姿（图中的黑色视点），输出在不同位姿（图中的蓝色视点）下渲染的新图片，以及在未来帧（时间帧 $t \in [T, T + i]$ ）的不同位姿（绿色视点）下的新图片。

受到Scaffold-GS中基于锚点的空间划分和SC-GS中基于控制点的空间划分的启发，我们采用一种更密集的空间划分方式：八叉树。具体来说，我们考虑对整个场景空间平均划分成八个部分（对应着三维坐标系的八个象限），根据场景重建精度的要求，我们可以让这划分除了的八个部分继续划分，从而构建一个八叉树。和普通的八叉树不同，我们构建的是一颗动态的八叉树，即每一个树中的节点，我们并不会固定其位置，而是令节点的位置在训练中出现一定的变化。这是因为在一个动态场景中，存在着高频和低频的区域，在高频区域中，往往需要更多的节点进行拟合，我们将固定的八叉树修改成动态的八叉树，这正是为了适应4D场景频率分布不同的情况。另外我们发现随着场景重建的进行，有一些子节点会随着位移的叠加，出现恶性优化的情况。具体来说，子节点会严重偏离其父节点的位置。父节点担任了更大区域的拟合任务，而子节点作为其细节的补充，父子节点距离过大往往使得这种需求无法得到实现。于是我们提出了父子节点的惩罚损失，在父子节点的距离过小的时候，该损失几乎不会发挥任何作用，而在父子节点距离过大的时候，该损失会对父子节点的距离惩罚，从而使得子节点的存在能够父节点的掌控区域进行细节上的补充。

最后，我们的主要贡献总结如下：

1. 我们提出了动态八叉树的空间划分方法，避免了基于控制点的方法导致的偏移问题，使得场景重建和预测不容易出现过拟合的情况。
2. 我们提出了节点的自由运动，保证八叉树的节点能够在一定程度上自由移动到欠拟合区域，保证空间中所有区域的稳定重建。
3. 我们提出了父子节点的距离惩罚损失，解决了父子节点距离过大的问题，使得子节点能够补充父节点无法精确重建的高频区域。
4. 在广泛应用的D-NeRF数据集上，我们在未来场景的新视图合成上，实现了更高质量的渲染结果。

Related Work

4D新视图合成

基于 NeRF 的动态隐式NVS是动态场景NVS领域早期的主流技术路线。早期工作D-NeRF首次将MLP作为变形场，建立空间坐标与时间戳到形变偏移的映射，实现了简单刚性动态场景的隐式重建；HyperNeRF通过引入时间维度扩展神经辐射场（NeRF）的隐式表征能力，实现了动态场景的时空几何建模，实验证实该方法能有效处理一定程度的非刚性形变场景；为提升变形场建模效率与泛化能力，HexPlane通过6平面分解实现了高效的动态变形函数建模，降低了计算开销；TiNeuVox结合平面与多网格的体素表征，进一步提升了动态隐式场景的训练与渲染速度。

3DGS在静态场景中展现出的训练效率与渲染质量优势，使其迅速被拓展至动态场景NVS领域，成为当前动态显式NVS的主流技术路线。。4DGS首次引入多尺度6平面特征分解与MLP解码器作为变形场，实现了高斯基元的时序变形建模，成为动态3DGS的基准方法；Deformable-GS通过深层MLP优化了动态场景的高斯基元变形；Grid4D提出4D网格特征分解，并使用哈希编码来压缩特征向量，提升了变形场对高频细节的建模能力，缓解了多尺度6平面特征分解带来的过平滑问题。4D-Rotor通过高维几何方法构造4D高斯基元，为动态场景建模提供了全新几何视角；Gaussian-flow通过粒子流函数拟合高斯基元时序运动，保证了局部变形的时空一致性；DynMF通过神经运动分解，提升了动态场景NVS的实时性与细节保真度；Motion-GS还额外引入了光流作为场景重建的外部监督，提高了基元运动的准确性。针对动态场景的不同约束，现有研究提出了多项针对性方案：SC-GS引入锚点表征实现了动态场景的可编辑NVS；DN-4DGS结合时空特征聚合优化了低质量输入视频的重建效果；Deblur4DGS解决了动态模糊场景下的建模难题；HiCoM实现了流式动态场景的在线NVS。

动态场景预测

在给定的现有场景观测中生成未来场景，在路径规划、智能决策中具有不可估量的价值。在深度学习的早期，就有结合3D卷积神经网络、长短期记忆网络或者transformer的未来场景估计方法，这些方法可以通过深度神经网络整合空间和时间上的信息。XX提出使用基于变分编码器的方法和XX提出了基于生成对抗网络的方法也利用了空间和时间在深度神经网络的整合来对未来场景进行预测。此外，利用更加先进的技术：扩散模型也可以达到类似效果。而基于图神经网络的方法在提供快速预测速度的同时表现出优异的泛化能力，因此本文考虑在此基础上实现动态场景预测。

3DGS动态场景预测

GuassianPrediction结合现有的动态预测算法和先进的3DGS技术，实现了3DGS动态场景的未来帧预测。其使用基于控制点的方法，把控制点当成点云，并使用GCN根据历史点云来预测未来点云。ODE-GS在前者的基础上，提出使用神经微分网络作为预测网络，优化了GCN无法精准预测动态连续运动的问题。

基于上述各类方法的限制，相比之下，我们提出的八叉树空间划分的方法，在基于GCN的未来场景预测中，有着高质量的新视图合成和准确的未来帧预测。

预备知识

在3DGS中，三维场景由一组独立的3D高斯基元集合 $\mathcal{G} = \{G_i\}_{i=1}^N$ 表示，其中 N 为场景中3D高斯基元的总数。对于第 i 个3D高斯基元 G_i ，其三维高斯分布定义为：

$$G_i = e^{-\frac{1}{2}(x-\mu_i)^T \Sigma_i^{-1}(x-\mu_i)}$$

式中， $\Sigma_i \in R^{3 \times 3}$ 为3D高斯基元的协方差矩阵，由尺度因子与旋转矩阵共同计算得到： $\Sigma_i = R_i S_i R_i^T$ ，其中 $S_i = \text{diag}(s_i^2)$ 为尺度对角矩阵，每个3D高斯基元都通过参数 $\{G_i : \mu_i, s_i, q_i, c_i, \sigma_i\}$ 来表示，分别代表该3D高斯基元的位置、尺度、旋转、颜色和不透明度，其中，旋转 q_i 使用4元数表示，颜色 c_i 通过3阶球谐函数解码得出

然后将这些3D高斯点投影到2D渲染平面上，给定一个视图变换 W 我们可以得到2D的协方差矩阵 Σ' 和2D高斯的中心 μ' ：

$$\Sigma' = JW\Sigma W^T J^T, \mu' = JW\mu$$

其中 J 是射影变换的仿射近似的雅可比矩阵。然后在2D渲染平面上的每一个像素点，检测每一个像素点是否落在一个2D高斯的3倍方差内，若是则视为该像素点触碰到了该2D高斯，然后根据体渲染公式进行颜色混合。具体来说，对于每一个像素点所触碰到的2D高斯，从前往后一共 N 个，同时所得到的颜色为 c_i ，渲染不透明度（包括高斯点本身的不透明度和像素点所在处的密度）为 α_i ，根据体渲染公式可以得到像素颜色 C ：

$$C = \sum_{i=1}^N c_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j)$$

Method

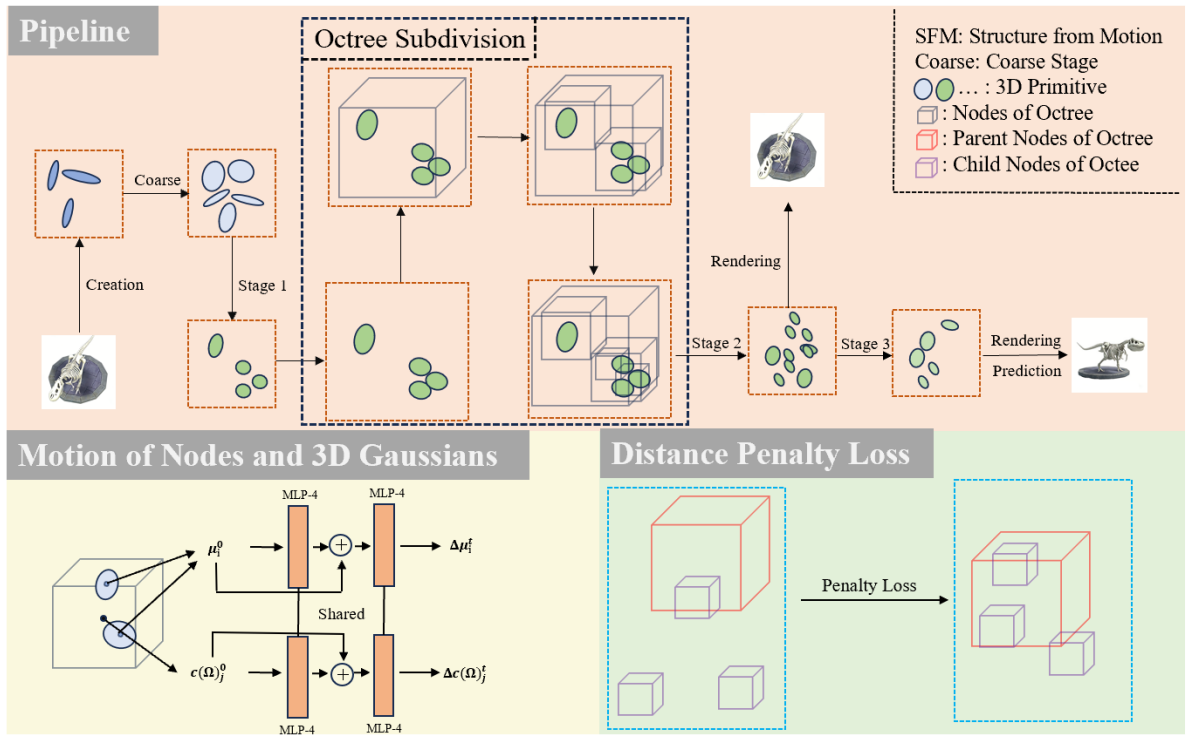


图2：本图系统展示了本文提出的OctGaussian方法的完整技术架构与核心设计，整体由三大深度关联的模块（Pipeline，Motion of Nodes and 3D Gaussians，Distance Penalty Loss）构成。1) Pipeline模块展示了完整的重建、新视图合成和未来场景预测的全流程，该流程通过数据集的初始点云和位姿作为输入，先通过初始点云创建初始的高斯场，并通过粗阶段、阶段1的动态场景重建阶段，然后对重建出来的高斯场进行基于Octree的空间划分（Pipeline模块中的Octree Subdivision子模块），以通过检测高斯点的密度的方式来决定空间的划分。最后通过阶段2的动态场景后优化阶段以及阶段3的未来场景预测网络训练，最后可以得到各个时间帧（包括未来时间帧）的新视图合成结果；2) Motion of Nodes and 3D Gaussians模块展现了基于MLP变形网络的节点与3D高斯点的变形策略，通过将第0帧的节点和3D高斯点的位置输入MLP变形网络可得到时间 t 下变形结果；3) Distance Penalty Loss模块展现了我们提出的父子节点的距离惩罚损失，通过该损失，我们可以在一定程度上限制子节点的运动，以避免过拟合。

给定一组图像，这些图像由不同的时间帧和不同位姿的相机拍摄，我们需要根据这组图像来预测未来时间帧下的不同位姿的新视图合成。如图2所示，展示了OctGaussian的Pipeline流程。首先我们需要输入一组包含时间帧和位姿的图像，通过SFM算法得到初始点云并进行粗阶段的场景重建。随后我们基于Octree的数据结构进行划分（4.1节）和3D高斯点与节点的运动的方法（4.2节）。然而各级节点的自由运动可能导致我们无法保持Octree中父子节点的关系，因此我们提出父子节点距离惩罚损失来限制各级节点的运动（4.3节）。

基于Octree的空间划分方式

在三维欧氏空间 $\mathbb{R}^3$ 中，我们将3D高斯点所在的空间区域设置为一个有界的立方体，我们将其记为根节点空间：

$$\Omega_0 = [X_{\min}, X_{\max}] \times [Y_{\min}, Y_{\max}] \times [Z_{\min}, Z_{\max}] \subset \mathbb{R}^3$$

然后将Octree的节点定义为一个立方体区域 $\Omega \subset \mathbb{R}^3$ ，其边长 $l(\Omega)$ 和中心点 $c(\Omega)$ 为：

$$l(\Omega) = (X_{\max} - X_{\min}, Y_{\max} - Y_{\min}, Z_{\max} - Z_{\min})^T \in \mathbb{R}^3$$

$$c(\Omega) = \left(\frac{X_{\max} + X_{\min}}{2}, \frac{Y_{\max} + Y_{\min}}{2}, \frac{Z_{\max} + Z_{\min}}{2} \right) \in \mathbb{R}^3$$

若要对Octree的节点进行划分，我们可以根据该节点的中心位置构建一个3维笛卡尔坐标系，然后根据其八个象限对父节点的立方体区域进行划分，最终我们可以得到8个子节点。

有了八叉树的构建方法，接下来，我们将根据场景中基元的密度来决定是否划分一个节点的立方体空间。我们将三维场景中的3D高斯点 $\mathcal{G} = \{G_i\}_{i=1}^N$ 的中心点单独分离出来构成一个集合 $\mathcal{P} = \{p_i = (x_i, y_i, z_i)^T \in \mathbb{R}^3\}_{i=1}^N$ 。对于任意一个立方体区域 Ω ，定义其内部的点云密度为：

$$\rho(\Omega) = \frac{|\mathcal{P} \cap \Omega|}{V(\Omega)}$$

其中 $|\mathcal{P} \cap \Omega|$ 表示落在 Ω 内部的点的个数，我们假设落在边界的点也属于 Ω 的内部点。 $V(\Omega)$ 为 Ω 的体积，对于立方体有 $V(\Omega) = l_x \times l_y \times l_z$ 。

有了立方体区域的3D高斯点密度，我们就可以根据密度阈值来决定是否对当前立方体区域进行划分。然而我们考虑到在不同的场景，场景的不同时间帧下，空间中的3D高斯点的数量各有不同，这对确定阈值比较困难。因此我们考虑采用相对密度：

$$\rho_{rel}(\Omega) = \frac{|\mathcal{P} \cap \Omega|/V(\Omega)}{|\mathcal{P}|/V(\Omega_0)}$$

即当前区域密度与全局平均密度之比。

最后，我们为场景划分设立一个密度阈值参数 ρ_{thr} 和一个八叉树的最大深度 D_{octree} ，当当前立方体区域的密度 $\rho_{rel}(\Omega) > \rho_{thr}$ 和节点层级 $K_{level} \leq D_{octree}$ 时，则划分子节点。我们设立最大深度是为了限制八叉树节点的数量，对于那些密度特别高但是由于最大深度限制无法继续划分的区域，我们在后续4.2节中介绍了节点的自由运动，这保证节点能够覆盖到需要覆盖的区域。

3D高斯点与节点的运动

3D高斯点的运动和节点的运动均通过变形场 Ω 来构建。一个变形场通常由一个神经网络的表示，该神经网络可以由多种方式构建，比如通过6平面分解方法、多层感知机（MLP）和4D超空间分解。这里我们选择使用MLP来作为变形场，其结构较为简单，且不易过平滑。给定一个基于MLP的变形场网络 D ，我们可以变形3D高斯点的位置与旋转以及节点的中心位置：

$$\begin{aligned} \Delta\mu_i^t, \Delta q_i^t &= D(\gamma(\mu_i^0), m_i, \gamma(t)) \\ \Delta c(\Omega)_j^t &= D(\gamma(c(\Omega)_j^0), m_i, \gamma(t)) \end{aligned}$$

这里的 $\gamma(\cdot)$ 我们设置为具有频率 L 的位置编码：

$$\gamma(x) = (\sin(2^l \pi x), \cos(2^l \pi x))_{l=0}^{L-1}$$

因此时间 t 下的3D高斯点的位置与旋转以及节点的位置为：

$$\mu_i^t = \mu_i^0 + \Delta\mu_i^t, \quad q_i^t = q_i^0 \otimes \Delta q_i^t$$

其中， $\otimes$ 代表关于4元数的乘法。

最后，为了防止变形场的训练进入局部最优解，以及为了模拟场景中物体拓扑变化的情况，我们遵循 Ω 提出的退火噪声与3D高斯点的生命周期。

父子节点的距离惩罚损失

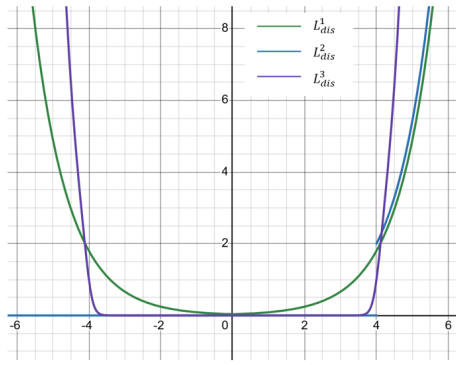


图3：本图展示了三种不同的距离惩罚损失函数的图像。相比于 L^1_{dis} 和 L^2_{dis} 损失，我们最终改善的 L^3_{dis} 损失既保持了连续性，有保证在一定的区间范围内值接近0，以避免距离惩罚损失阻碍节点的优化。

由于节点是可以自由运动的，当父节点和子节点之间的距离过大的时候，就会破坏八叉树的结构，因此我们需要对子体素距离父体素过远的时候进行惩罚。对于惩罚函数，我们可以使用基于指数函数的构造方法限制了节点的位置优化。但是该方法在合法优化范围内仍然存在大量惩罚。而我们的目的是能够给予子节点一定的自由度，但是又不允许其离开父节点太远，于是可以借助tanh函数来构造近似的损失函数，使其能够在一定的距离范围内自由移动，但是超过了这个距离就会受到巨额惩罚。

具体来说，我们最先考虑了使用指数函数作为惩罚：

$$L^1_{dis} = ke^{|x|-M}$$

这里的 M 为临界值，是一个基于场景大小的预设值， k 为函数的幅度， x 为父子节点的距离。该损失函数如图X所示，其在 $[-M, M]$ 的范围内仍然存在一定惩罚，这无疑干扰了正常的变形场优化。

为此我们还考虑使用分段函数来进行惩罚：

$$L^2_{dis} = \begin{cases} 0, & x < M \\ ke^{|x|-M}, & x \geq M \end{cases}$$

这消除了节点在 $[-M, M]$ 范围移动受到限制的问题。但是这破坏了函数的连续性，且在断点处惩罚的突然增大在一定程度阻碍了节点运动的优化。综上，我们借助tanh函数来构造一个函数：

$$L^3_{dis} = \left(\frac{|x|}{M}\right)^g \cdot \left(1 + \tanh\left(\tau \cdot \left(\frac{|x|}{M} - 1\right)\right)\right)$$

其中，这里的 τ 为转折点的陡峭程度， g 为增长速率。通过该损失函数，在该损失下，我们既保证了连续性，又保证了父子节点在合理范围内不会受到过量惩罚。如图3所示，我们展示了三种不同损失函数的图像，其中 L^1_{dis} 中的 k 取2， L^2_{dis} 中的 k 取1.8（为了避免函数图像重叠，影响观察）， M 取4， g 取10， τ 取20。

训练细节

实验

本节通过在多个场景下的定量对比实验、定性可视化分析与消融实验，系统性验证本文提出的OctGaussian方法在动态场景及未来帧的新视图合成任务中的有效性、优越性与泛化性。所有实验均在18 vCPU AMD EPYC 9754 128-Core Processor，内存60G，以及单张 RTX 4090D GPU 24G上完成，系统版本为Ubuntu 22.04。渲染管线采用3DGS官网开源的3D高斯体渲染器实现。所有实验设置与基线方法保持对齐，确保对比的公平性与实验结果的可复现性。

实验设置

4.1.1 数据集 (Datasets)

本文在动态场景NVS领域广泛使用的 D-nerf合成数据集上完成所有核心实验。该数据集为使用blender生成的动态场景数据集，包含了8个动态场景，一共由100-200张训练图像和20张测试图像组成，所有图像的时间范围均在0到1之间。我们考虑使用 800×800 的分辨率的图像来训练和渲染。我们遵循[1]的实验比较设置，统一使用黑色背景来评估渲染结果。

4.1.2 基线方法 (Baselines)

为充分验证本文方法的优越性，我们选取了当下5种主流的动态场景NVS和动态场景预测方法作为基线。具体如下：

1. TiNeuVox：基于 NeRF 的动态场景NVS方法，采用平面与3D网格结合的体素表征，是隐式动态场景新视图合成领域的经典 SOTA 方法。
2. 4DGS：基于3DGS的动态场景新视图合成基准方法，采用多尺度6平面特征分解框架构建变形场，是当前动态场景新视图合成领域的主流标杆方法。
3. Deformable-GS：基于3DGS的动态场景新视图合成基准方法，采用MLP构建变形场，是当前动态场景新视图合成领域的主流标杆方法。
4. GaussianPrediction：基于3DGS和GCN动态预测的动态场景预测方法，使用基于控制点的方法来构建3D场景，并使用GCN作为动态场景预测的主要网络，是当前动态场景及未来场景的新视图合成的主流标杆方法。
5. ODE-GS：基于3DGS和ODE网络的动态场景预测方法，其使用ODE来拟合控制点的连续运动，并能够预测未来控制点的运动，是当前动态场景及未来场景的新视图合成的主流标杆方法。

4.1.3 超参数设置

本文方法的核心超参数设置与GaussianPrediction基线保持对齐，确保对比的公平性。对于所有数据集，我们的渲染相关参数、致密化策略、变形场的网络结构、学习率、优化策略等核心训练参数，与GaussianPrediction的官方设置保持对齐。同时，我们的方法的超参数设置为 $\rho_{thr} = 0.1 \cdot |\mathcal{P}|/V(\Omega_0)$, $D_{octree} = 3$, $M = 0.1 \cdot \mathcal{I}(\Omega_0)$, $g = 10$, $\tau = 20$ 。所有实验均采用端到端的训练方式，无额外的后处理步骤。

4.2 与SOTA方法的对比实验

4.2.1 定量结果分析

Table 1: Quantitative motion prediction results comparison on D-NeRF dataset. The higher PSNR ($\uparrow$), higher SSIM ($\uparrow$), and lower LPIPS ($\downarrow$) denote better rendering quality. The color of each cell shows the **best** and the **second best**.

Method	bouncingballs			hellwarrior			hook			jumpingjacks		
	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS
TiNeuVox	25.93	0.9665	0.0832	29.36	0.9097	0.1132	24.40	0.9163	0.0506	20.98	0.9198	0.0751
4DGS	29.57	0.9752	0.0431	31.42	0.9263	0.0821	23.92	0.9133	0.0515	20.53	0.9164	0.0789
Deformable-GS	28.94	0.9781	0.0278	29.31	0.0838	0.0834	23.88	0.9137	0.0536	20.57	0.9174	0.0787
GaussianPrediction	29.03	0.9802	0.0225	29.91	0.9099	0.0735	23.85	0.9128	0.0516	20.61	0.9167	0.0787
ODE-GS	28.41	0.9765	0.0630	29.02	0.9091	0.0725	24.43	0.9188	0.0481	20.47	0.9170	0.0793
Ours	28.71	0.9795	0.0271	31.49	0.9329	0.0655	24.24	0.9158	0.0502	20.92	0.9208	0.0740

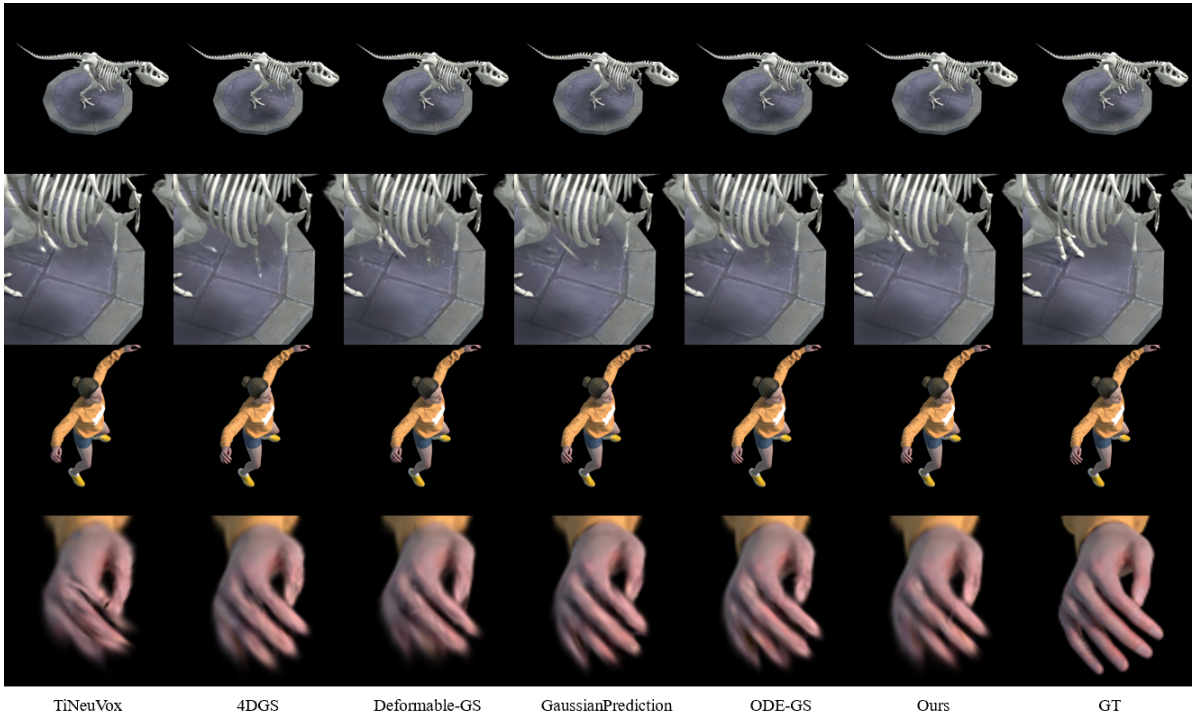
Method	mutant			standup			trex			mean		
	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS
TiNeuVox	24.43	0.9281	0.0731	26.50	0.9462	0.0441	21.46	0.9453	0.0431	24.72	0.9331	0.0689
4DGS	24.68	0.9263	0.0573	26.83	0.9470	0.0447	21.44	0.9451	0.0413	25.48	0.9357	0.0570
Deformable-GS	24.35	0.9310	0.0442	25.71	0.9440	0.0463	21.41	0.9430	0.0438	24.88	0.8159	0.0540
GaussianPrediction	25.79	0.9360	0.0296	26.97	0.9471	0.0440	21.74	0.9455	0.0414	25.41	0.9355	0.0488
ODE-GS	26.35	0.9452	0.0276	27.16	0.9477	0.0419	20.50	0.9367	0.0508	25.19	0.9359	0.0547
Ours	28.42	0.9580	0.0236	27.93	0.9505	0.0380	20.36	0.9360	0.0526	26.01	0.9419	0.0473

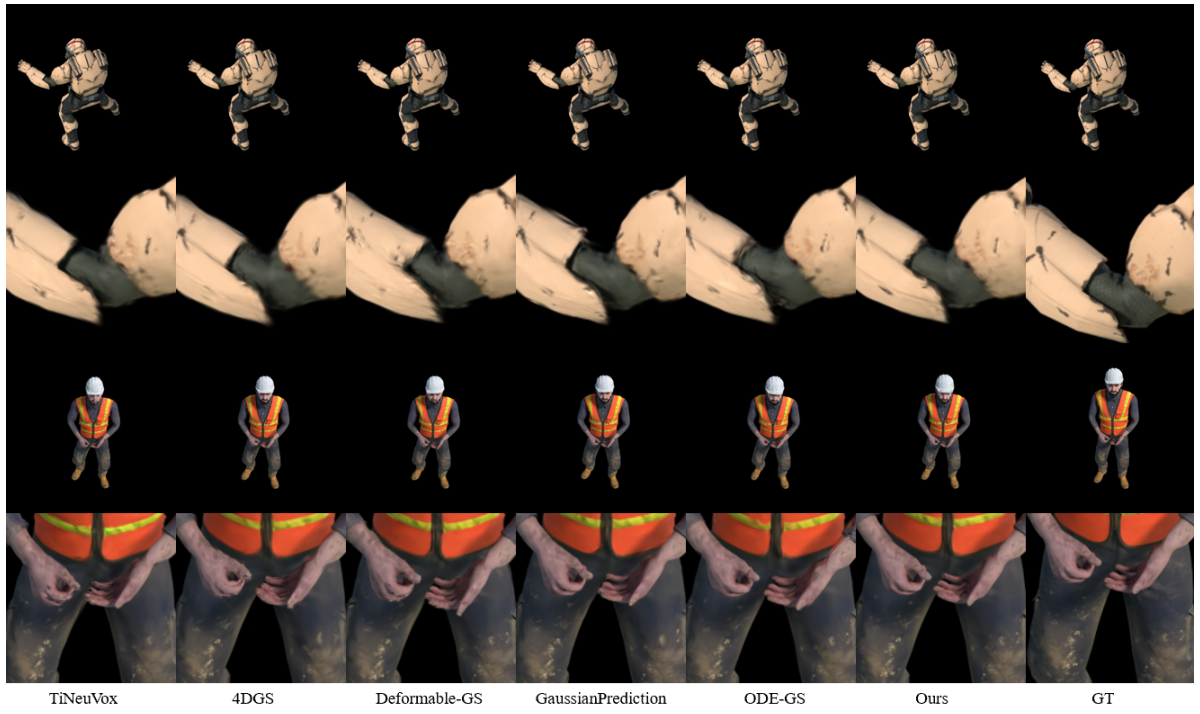
Table 2: Quantitative rendering results comparison on D-NeRF dataset. The higher PSNR ($\uparrow$), higher SSIM ($\uparrow$), and lower LPIPS ($\downarrow$) denote better rendering quality. The color of each cell shows the **best** and the **second best**.

Method	bouncingballs			hellwarrior			hook			jumpingjacks		
	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS
TiNeuVox	37.28	0.9837	0.0392	39.25	0.9773	0.0501	41.47	0.9942	0.0101	36.03	0.9892	0.0144
4DGS	41.32	0.9912	0.0154	39.21	0.9781	0.0525	42.94	0.9952	0.0070	39.19	0.9936	0.0093
Deformable-GS	43.60	0.9953	0.0092	44.46	0.9825	0.0221	42.03	0.9945	0.0078	38.55	0.9930	0.0102
GaussianPrediction	45.71	0.9954	0.0072	46.32	0.9883	0.0131	43.08	0.9959	0.0066	37.62	0.9922	0.0107
ODE-GS	45.36	0.9951	0.0071	47.81	0.9901	0.0172	43.04	0.9952	0.0067	38.84	0.9933	0.0094
Ours	46.86	0.9974	0.0067	48.16	0.9948	0.0128	43.46	0.9956	0.0061	39.46	0.9939	0.0091

Method	mutant			standup			trex			mean		
	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS
TiNeuVox	35.21	0.9771	0.0308	44.35	0.9948	0.0097	34.61	0.9912	0.0139	38.31	0.9868	0.0240
4DGS	39.41	0.9862	0.0251	45.31	0.9957	0.0075	37.97	0.9941	0.0099	40.76	0.9906	0.0181
Deformable-GS	43.25	0.9951	0.0063	45.56	0.9959	0.0072	36.96	0.9931	0.0114	42.06	0.9928	0.0106
GaussianPrediction	44.71	0.9954	0.0042	45.35	0.9959	0.0074	36.59	0.9932	0.0115	42.77	0.9938	0.0087
ODE-GS	43.53	0.9954	0.0041	45.44	0.9959	0.0072	37.08	0.9932	0.0111	43.01	0.9940	0.0090
Ours	45.63	0.9976	0.0035	46.19	0.9964	0.0066	38.53	0.9943	0.0098	44.04	0.9957	0.0078

4.2.2 定性结果分析





本文方法与基线方法的可视化渲染对比如上图[3]图[4]所示所示，分别对应NVS任务与预测NVS任务的渲染结果。从NVS任务的可视化结果（图 3）可以看出，以往的方法均存在明显的边缘模糊问题，在动态物体的轮廓区域出现了不同程度的细节丢失与渲染伪影。比如 GaussianPrediction方法中基于控制点的划分方式无法层次化地覆盖到欠拟合位置，在快速运动区域的平滑部分出现黑色的伪影，ODE-GS方法虽能还原场景主体结构，但在运动物体边缘出现了明显的模糊与时序抖动。而本文方法能够精准重建并渲染动态物体的边缘轮廓，且不会出现伪影，在恐龙骨架的脚部和手部、任务的手指部分等细微动态区域，均实现了清晰的细节还原，无明显的时序抖动与结构模糊。在难度更高的预测NVS任务中（图 4），本文方法的性能优势更为突出。在手臂运动中，本文方法能够清晰重建并渲染盔甲边缘等高频细节，而以往均出现了不同程度的细节模糊与高频伪影；在standup场景中，本文方法不仅精准还原了手指和手臂处的纹理，且不会产生额外的高频细节，而以往的方法完全丢失了手臂的纹理细节和手指边缘的高频信息。上述可视化结果与定量指标的变化趋势完全一致，充分验证了本文提出的 GaussianPrediction 方法显著提升了动态场景下NVS任务和预测NVS任务的能力，实现了比现有主流 SOTA 方法更优的视觉保真度与时序一致性。

4.3 消融实验和分析

Table 4: Quantitative ablation comparison results for the prediction task on the D-NeRF dataset. The higher PSNR ($\uparrow$), higher SSIM ($\uparrow$), and lower LPIPS ($\downarrow$) denote better rendering quality. The color of each cell shows the **best** and the **second best**.

Method	PSNR	SSIM	LPIPS
w/o oct	25.45	0.9298	0.0520
w/o mot	21.32	0.9041	0.1247
w/o los	25.73	0.9380	0.0481
Ours	26.01	0.9419	0.0473

Table 5: Quantitative ablation comparison results for the NVS task on the D-NeRF dataset. The higher PSNR ($\uparrow$), higher SSIM ($\uparrow$), and lower LPIPS ($\downarrow$) denote better rendering quality. The color of each cell shows the **best** and the **second best**.

Method	PSNR	SSIM	LPIPS
w/o oct	42.73	0.9935	0.0082
w/o mot	33.86	0.9690	0.0291
w/o los	44.01	0.9902	0.0080
Ours	44.04	0.9957	0.0078

5 结论
